OS ORANGE ASSIGNMENT 2

NAME: ANVII S RAI

SRN: PES2UG24CS076

SECTION: 4B

Phase 1

./test_objects

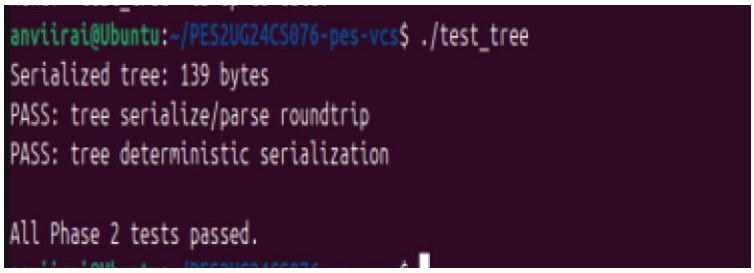
```
anviirai@Ubuntu:~/PES2UG24CS076-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
anviirai@Ubuntu:~/PES2UG24CS076-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

`find .pes/objects -type f`

```
anviirai@Ubuntu:~/PES2UG24CS076-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
anviirai@Ubuntu:~/PES2UG24CS076-pes-vcs$
```

PHASE 2

./test_tree



find .pes/objects -type f

xxd .pes/objects/d7/be03b9c3a0f000d92dd1c16f51731c647228d39b4e7a5f171a1093af6c1156 | head -20

PHASE 3

pes init → pes add → pes status

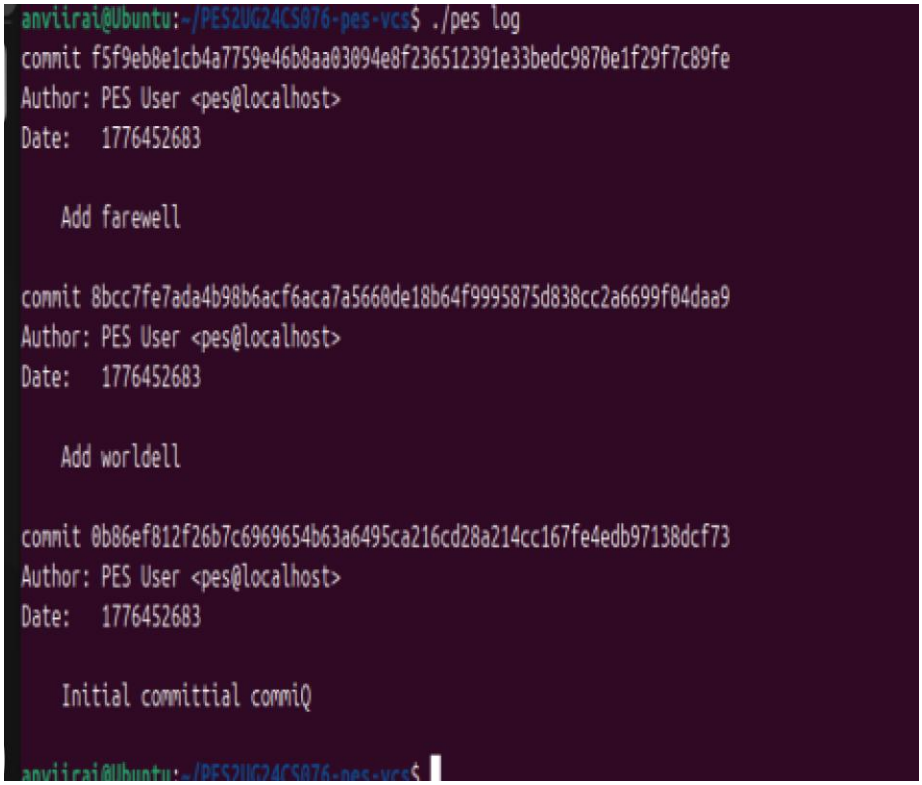
```
anviirai@Ubuntu:~/PES2UG24CS076-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
anviirai@Ubuntu:~/PES2UG24CS076-pes-vcs$ echo "Hello PES" > test.txt
./pes add test.txt
anviirai@Ubuntu:~/PES2UG24CS076-pes-vcs$ ./pes status
Staged changes:
  staged:     test.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  index.h
  untracked:  .gitignore
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  Makefile
  untracked:  test_tree
  untracked:  object.c
  untracked:  test_objects.c
  untracked:  index.c
  untracked:  test_sequence.sh
  untracked:  pes.h
  untracked:  tree.h
  untracked:  README.md
  untracked:  pes.c
  untracked:  commit.h
  untracked:  commit.c
```

cat .pes/index

```
anviirai@Ubuntu:~/PES2UG24CS076-pes-vcs$ cat .pes/index
◆◆◆◆◆◆.=◆(◆>G$◆◆◆v◆◆\◆.tb◆◆x{◆i
anviirai@Ubuntu:~/PES2UG24CS076-pes-vcs$ xxd .pes/index
00000000: 0100 0000 b481 0000 f4e3 b312 fbd4 2e3d  ...............=
00000010: 1a28 a619 3e47 24ff 95a3 76e2 1b7c 845c  .(..>G$...v..|.\
00000020: d52e 7413 62c7 cc78 0000 0000 7f7b e269  ..t.b..x.....{.i
00000030: 0000 0000 0a00 0000 7465 7374 2e74 7874  ........test.txt
00000040: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000050: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000060: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000070: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000080: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000090: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000a0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000b0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000c0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000d0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000e0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
000000f0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000100: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000110: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000120: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000130: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000140: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000150: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000160: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000170: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000180: 0000 0000 0000 0000 0000 0000 0000 0000  ................
```

PHASE 4

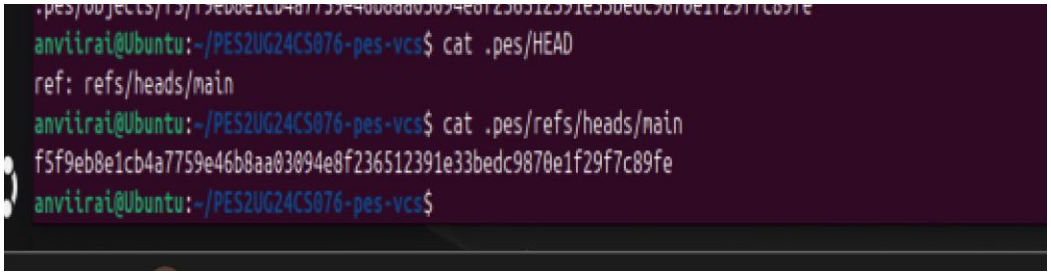
./pes log



find .pes/objects -type f | sort

cat .pes/HEAD

cat .pes/refs/heads/main



Phase 5: Branching and Checkout

Q5.1: Implementing pes checkout <branch>

- **Files in .pes/ to change:** The main file to update is .pes/HEAD. It needs to change from pointing at the current branch (like ref: refs/heads/main) to the target branch (like ref: refs/heads/dev).
- **Working Directory:** This is where the actual work happens. The program must look at the "Tree" hash stored in the target branch commit and extract those files into the actual folder.
- **Complexity:** This is hard because of **safety**. If a user has unsaved work in the folder that hasn't been committed, a naive checkout would just overwrite it. The logic must check for local changes before touching any files to avoid data loss.

Q5.2: Detecting a "Dirty" Directory To find a conflict, you compare three specific hashes for every file:

1. **The file on disk:** The actual version the user is looking at.
2. **The Index:** The version the user last "added."
3. **The Target Tree:** The version of the file in the branch being checked out. If the hash of the file on disk is different from the hash in the Index, the directory is "dirty." If that file also differs between the current branch

and the target branch, the checkout must stop to prevent the user's uncommitted work from being erased.

Q5.3: Detached HEAD State

- **What happens:** In this state, HEAD points to a specific commit hash instead of a branch name. You can still make new commits, and HEAD will move forward normally to the new hashes.
- **The Risk:** Since no branch (like main) is "holding" these commits, they have no name. If you switch back to main, the address of those commits is lost.
- **Recovery:** A user can look at the reflog (a log of where HEAD has been) to find the lost hash and then run git branch <name> <hash> to give that commit a permanent name.

Phase 6: Garbage Collection (GC)

Q6.1: The Cleanup Algorithm

- **Algorithm:** This uses a **"Mark and Sweep"** strategy.
    1. **Mark:** Start at every branch tip and tag. Follow every link from those commits to their trees and blobs. Every hash found this way is marked as "reachable."
    2. **Sweep:** Scan the .pes/objects folder. Any file whose hash is not on the "reachable" list is deleted.
- **Data Structure:** A **Hash Set** is the most efficient choice because checking if a specific hash exists in the "reachable" list is very fast.
- **Estimation:** For 100,000 commits and 50 branches, the algorithm must visit every commit in the history. Even if most commits share files, the process involves checking **hundreds of thousands of objects** to ensure nothing important is deleted.

Q6.2: GC Race Conditions

- **The Danger:** A race condition happens if GC runs while a user is in the middle of a commit command.
- **Scenario:** The user runs add, which writes a new blob to the disk. Before the commit command finishes writing the commit object that points to that blob, the GC starts. The GC sees a blob that nothing points to and deletes it. The commit then finishes, but it points to a file that no longer exists.
- **The Fix:** Git avoids this by using a **grace period**. The GC will only delete unreachable objects that are older than a certain time (usually 2 weeks). This ensures that any "in-progress" objects are safe.